

תהליך פיתוח מונחה-AI לתכנית ההאצה GovLabs, מבית שטראוס אסטרטגיה: ה-Agents מובילים, ה-Skills מקנים את הידע, הצוות מכריע ומאשר — מהחבילה העסקית ועד קוד בדוק, עם ה-Governance שממשלה דורשת.

1. התפיסה

GovLabs AI-SDLC הוא מודל הפעלה חדש לפיתוח מיזמי GovLabs, שנבנה סביב היכולות שה-AI מביא: לקרוא הקשר רחב, לנסח תוצרים מלאים, לזהות פערים ולשאול, לבדוק את עצמו ולעבוד ברציפות. במקום להוסיף AI לתהליך קיים, התהליך עצמו מתוכנן מחדש כך שה-AI מוביל אותו.

ההובלה נעשית על-ידי סוכני AI (Agents), והידע מגיע ממיומנויות (Skills). סוכן מנצח (Conductor) מכיר את המתודולוגיה ואת מצב המיזם ומריץ אותו קדימה: פותח כל שלב, אוסף את הקלט, מפעיל את הסוכן המתמחה בשלב (מוצר, ארכיטקטורה, בנייה, בדיקות), מריץ בדיקות, מזמן סוקר, מציג לאישור, מתעד את הלקחים ומכין את השלב הבא. הידע המקצועי — איך כותבים דרישות, איך מתעדים החלטה ארכיטקטונית, איך בונים תכנית בדיקות לפי תקני הממשלה — ארוז ב-Skills של GovLabs שהסוכנים מפעילים. השילוב הוא הליבה: ה-Agent הוא הביצוע, ה-Skill הוא המומחיות.

הצוות האנושי עובר לתפקיד של הכוונה, הכרעה ואישור. המנטור העסקי וצוות המיזם מגדירים את הכוונה וקובעים סדרי עדיפויות; הארכיטקט בוחר בין חלופות ומאשר החלטות; צוות ה-IT של המשרד קובע אילוצים; המפתחים והבודקים מכריעים מה מוכן ומה חוסם. הזמן של כולם מושקע במה שרק הם יכולים לתת — הקשר, שיקול דעת וסמכות.

ההאצה נובעת מכך שה-AI מוביל ומייצר: בכל שלב הסוכנים מגיעים עם התוצר כתוב — לא עם דף ריק — וכל תוצר מאושר הוא כבר הקלט המוכן של השלבים הבאים. הצוות לא כותב מסמכים ולא מתרגם בין שלבים; הוא מכריע על טיוטות מלאות, והסוכנים ממשיכים מיד עם ההכרעות. כך זמן ההמתנה בין שלבים — שהוא רוב הזמן בתהליך מסורתי — כמעט נעלם. ה-Governance מובנה בתהליך ולא נוסף עליו: כל תוצר הוא קובץ מפרט מנוהל-גרסאות; כל החלטת מוצר נרשמת ב-decisions.md וכל החלטה ארכיטקטונית ב-ADR; כל תוצר עובר בדיקה אוטומטית וסקירה של סוכן Reviewer; ואף שלב לא נסגר ללא אישור של בעל סמכות. שער האישור האנושי הוא מה שמאפשר להעניק ל-AI את ההובלה.

הידע מצטבר. כל מיזם קורא מ"מוח כללי" של GovLabs (Skills, כללים, תבניות, לקחים) ומחזיר אליו את מה שלמד. כל מיזם מתחיל מנקודה מתקדמת יותר מקודמו, והמתודולוגיה עצמה משתפרת עם כל הרצה.

2. הערך העסקי — למה בכלל כדאי לעבור ל-AI-Driven

ערך	איך התפיסה מייצרת אותו
מהירות	הסוכנים מייצרים את התוצרים המלאים של כל שלב, והצוות מכריע ומאשר במקום לכתוב. מפגשי צוות ממוקדים של שעות במקום סבבי תיאום של שבועות; מחזורי בנייה של ימים לכל סיפור משתמש. בשלב הראשון הצוות מפעיל את הסוכנים בעצמו מתוך הכלי שלו; בעתיד ניתן להעביר חלק מהריצה לאוטונומיה — התהליך והתוצרים לא משתנים.
איכות ופחות הפתעות	קריטריוני קבלה כתובים לכל דרישה לפני שנכתב קוד; כל דרישה ניתנת למעקב עד הבדיקה שמוכיחה אותה; סוקר ובדיקות אוטומטיות לפני כל שער.
Governance ושקיפות	שרשרת מלאה של מי החליט מה ומתי, מנוהלת-גרסאות. ADR לכל החלטה ארכיטקטונית. כללי אבטחה, פרטיות ונגישות ממשלתיים מובנים ב-Skills ובבדיקות — לא תלויים בזיכרון של אדם.
המשרד הוא הבעלים של הידע	המפרט, הארכיטקטורה וההחלטות הם נכס של המשרד — מסמכים חיים ברפו שלו, לא ידע שנמצא רק אצל הספק. החלפת ספק, הרחבה או תחזוקה מתחילות ממסמכים מעודכנים.
אחידות וסטנדרט	אותו תהליך, אותם תוצרים, אותם שערים, אותה שפה — לכלל המיזמים בתכנית. מנטור שליווה מיזם אחד מבין מיד את השני; הנהלת GovLabs רואה את כל המיזמים באותה שפה.
ידע שמצטבר	לקח ממיזם אחד הופך לכלל או ל-Skill משופר במוח הכללי ומשרת את כל המיזמים הבאים. העלות והסיכון של כל מיזם יורדים עם הזמן.
מוכנות לעתיד	תפיסה חדשנית ומתקדמת שאינה תלויה בכלי כזה או אחר: המתודולוגיה, ה-Skills והתוצרים עומדים בפני עצמם. מעבר לכלי חדש או לסוכנים אוטונומיים הוא שינוי במידת האוטונומיה, לא בתהליך.

3. אבני היסוד

Agent — מי מבצע 🤖

סוכן AI עם תפקיד מוגדר: פרסונה (למשל "ארכיטקט תוכנה שמכיר את תקני הענן הממשלתי"), הכלים שמותר לו להשתמש בהם (קריאה וכתיבה של קבצים, הרצת בדיקות, חיפוש), והמנדט שלו בתהליך (אילו שלבים הוא מוביל). הסוכן מנהל את השיחה עם הצוות: שואל, מציע, כותב, ומציג לאישור.

בתפיסה יש שבעה: **Conductor** (מנצח — מריץ את התהליך), **Product**, **Architect**, **API**, **Builder**, **Tester** ו-**Reviewer** (סוקר).

Skill — איך עושים את זה נכון 📖

חבילת ידע לשימוש חוזר: קובץ הוראות שמסביר מתי משתמשים בו, מהם השלבים, מה תבנית הפלט ואיך נראית דוגמה טובה — לעיתים עם תבניות וסקריפטים מצורפים. הסוכן מפעיל את ה-Skills שהוא צריך לשלב הנוכחי. אותו Skill משרת כל סוכן, כל מיזם וכל כלי.

ה-Agent הוא המבצע, ה-Skill הוא המומחיות. Skill חדש או משופר משדרג מיד את כל המיזמים — בלי לשנות את הסוכנים.

GovLabs Knowledge — המוח הכללי 🧠

מאגר אחד של GovLabs שכל מיזם מקבל: ספריית ה-Skills, כללי הממשלה (אבטחה, פרטיות, נגישות, ענן), תבניות לכל קובץ, דוגמאות טובות ולקחים ממיזמים קודמים. נטען לכל שלב, ומתעדכן ממה שנלמד. לצידו **זיכרון המיזם** — מה שפרטי למיזם אחד: ההקשר הקבוע שלו, ההחלטות, מצב ההתקדמות והלקחים של כל שלב.

4. שמונה עקרונות

1 ה-AI מוביל, הצוות שולט

סוכן מנצח מריץ את התהליך מקצה לקצה. הצוות מכוון, מכריע ומאשר. אף שלב לא נסגר בלי אישור של בעל סמכות.

2 המפרט הוא מקור האמת

כל תוצר הוא קובץ Markdown מנוהל-גרסאות ברפּו של המיזם. מה שלא כתוב במפרט — לא קיים.

3 הצוות כולו עובד יחד

מנטור עסקי, צוות המיזם, ארכיטקט, IT המשרד, מוצר, פיתוח ובדיקות — כל אחד בתפקידו — יושבים יחד מול ה-AI. שאלות נענות במקום, החלטות נרשמות במקום.

4 כל החלטה מתועדת

החלטות מוצר ב- `decisions.md`, החלטות ארכיטקטורה ב-ADR. כל דרישה ניתנת למעקב: מאיפה הגיעה, מי אישר, איפה נבדקה.

5 בודקים לפני כל שער

בדיקה אוטומטית (סעיפים חסרים, עקיבות, מידע אישי) + סוכן Reviewer. הצוות מקבל ממצאים, לא דף ריק.

6 מוח אחד לכל המיזמים

GovLabs Knowledge נטען לכל שלב בכל מיזם, ומתעדכן ממה שנלמד. כל מיזם מתחיל מנקודה מתקדמת יותר מקודמו.

7 עברית, מימין לשמאל

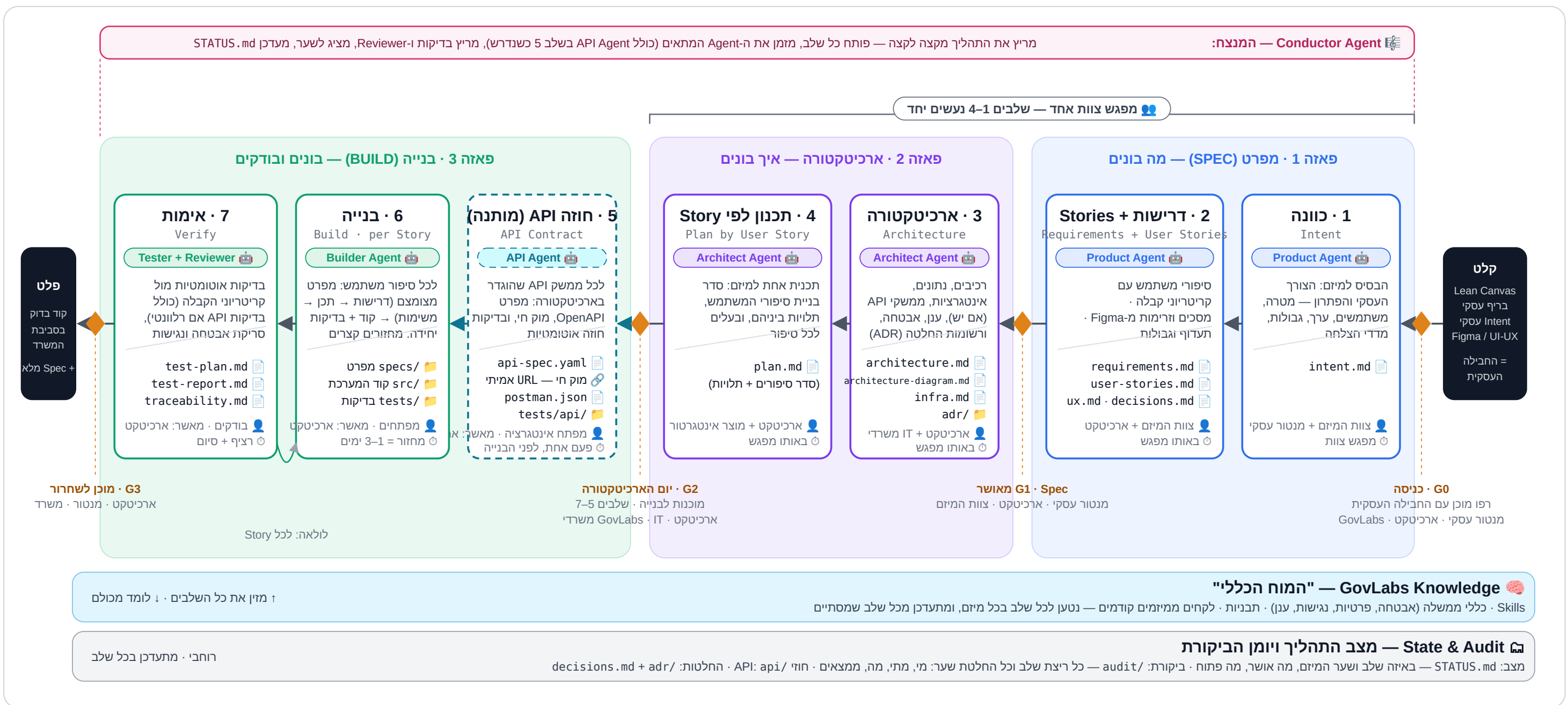
התוצרים והתהליך מוצגים לצוות בעברית. בפועל הסוכנים עובדים באנגלית — איכות גבוהה יותר, יעילות וחיסכון בטוקנים — ובסוף כל מסמך פרק "תרגום לעברית" שמחייב את הצוות.

8 אגנוסטיים לכלי, גדלים בהדרגה

המתודולוגיה, ה-Skills והתוצרים לא תלויים בכלי — Kiro, Antigravity, Claude Code או אחר. היום הסוכנים רצים כשהצוות מולם; מחר — אוטונומית על אותם קבצים. התהליך לא משתנה, רק מידת האוטונומיה.

5. ה-Pipeline — שלוש פאזות, שבעה שלבים, ארבעה Gates

הזרימה מימין לשמאל. ה-Conductor (למעלה) מריץ את כל התהליך; בכל שלב מסומן הסוכן המוביל. פאזות 1–2 (שלב 1–4) מתבצעות במפגש צוות אחד; שלב 5 (חווה API) הוא שלב מותנה שרץ פעם אחת, רק אם הארכיטקטורה מגדירה ממשקי API; שלבים 6–7 רצים במחזוריים קצרים לכל סיפור משתמש. שני הפסים למטה הם הרכיבים הרוחביים: המוח הכללי שמזין את כל השלבים, ו-State & Audit — מצב התהליך ויומן הביקורת שמתעדכנים בכל שלב.



התפקידים בתהליך

תפקיד	מה הוא נותן לתהליך
מנטור עסקי	בעל הכוונה העסקית; מאשר את הכוונה והדרישות ואת המוכנות לשחרור.
ארכיטקט (טכנולוגי)	הסמכות הטכנולוגית: בוחר בין חלופות, מאשר ADR, תכנית העבודה, הבנייה והאימות.
צוות המיזם	אנשי המשרד שמובילים את המיזם: עונים על שאלות הכוונה והדרישות, מתעדפים.
צוות GovLabs	מנחה את התהליך, מנהל את המוח הכללי, מאשר כניסה ומוכנות לבנייה.
צוות IT משרדי	אילוצי הסביבה, האבטחה והתשתית של המשרד; שותף לאישור הארכיטקטורה.
מוצר אינטגרטור	מפרט את סיפורי המשתמש ומסדר עדיפויות לבנייה.
מפתחי אינטגרטור	מאשרים מפרט סיפור לפני קוד, סוקרים קוד, מכריעים בשאלות מימוש.
בודקים	מובילים את שלב האימות מהצד האנושי: מאשרים את תכנית הבדיקות שנגזרה מהדרישות, משלימים בדיקות שהאוטומציה לא מכסה (קבלה, חוויית משתמש, תרחישי קצה), מאמתים את מטריצת העקיבות וממליצים לארכיטקט על מוכנות לשחרור.

#	שלב	קלט	Agent	Skills	הצוות	פלט	מאשר
0	שער כניסה G0	החבילה העסקית: Lean Canvas, בריף עסקי, Intent עסקי, עיצוב Figma / UI-UX	Conductor	status	צוות GovLabs יוצר את הרפו מהתבנית	רפו מוכן עם החבילה העסקית	מנטור עסקי, ארכיטקט, צוות GovLabs
1	כוונה	החבילה העסקית	Product	business-package-to-intent	צוות המיזם + מנטור עסקי עונים ומאשרים	intent.md — הבסיס למיזם: הצורך העסקי והפתרון	מנטור עסקי + צוות המיזם
2	דרישות + Stories	intent.md + עיצובי Figma	Product	write-requirements write-user-stories figma-to-ux	צוות המיזם, מוצר אינטגרטור, ארכיטקט: מתעדפים ומכריעים על גבולות	requirements.md user-stories.md ux.md decisions.md	מנטור עסקי + ארכיטקט
—	שער G1	המפרט מאושר: לכל סיפור משתמש (user-stories.md) קריטריוני קבלה, כל מסך מקושר לסיפור, אין שאלות פתוחות חוסמות. מאשרים: מנטור עסקי, ארכיטקט, צוות המיזם.					
3	ארכיטקטורה	הדרישות + אילוצי סביבת המשרד	Architect	architecture-doc architecture-diagram security-privacy-requirements adr gov-cloud-patterns	ארכיטקט + IT משרדי בוחרים בין חלופות ומאשרים כל ADR	architecture.md architecture-diagram.md infra.md adr/	ארכיטקט + IT משרדי
4	תכנון לפי Story	ארכיטקטורה + user-stories.md	Architect	story-plan	ארכיטקט + מוצר אינטגרטור מסדרים עדיפויות	plan.md — סדר בניית הסיפורים, תלויות, בעלים (אחד למיזם)	ארכיטקט
—	שער G2 • יום הארכיטקטורה	בדיקת מוכנות למעבר לשלבים 5–7: לכל סיפור משתמש יש דרישות, ארכיטקטורה וסדר בנייה; אם יש ממשקי API הם מזוהים ומתועדים. מאשרים: ארכיטקט, צוות IT, GovLabs משרדי.					
5	חוזה API (מותנה)	ממשקי API מהארכיטקטורה (אם יש)	API	api-contract-spec api-mock api-tests postman-collection	מפתח אינטגרציה מאשר את מפרט ה-API; ארכיטקט מאשר את השלב	api/spec.yaml מוק חי (URL) api/postman.json api/tests/	ארכיטקט
6	בנייה (לכל Story)	סיפור משתמש + הדרישות והארכיטקטורה שלו	Builder	story-spec coding-standards-gov adr (אם התגלתה החלטה)	מפתחי אינטגרטור מאשרים מפרט סיפור לפני קוד וסוקרים קוד	specs/<story>/ משימות src/ — קוד המערכת tests/ — בדיקות	ארכיטקט

7	אימות	קוד + דרישות (+ בדיקות ה-API אם רץ שלב (5)	Tester Reviewer	test-plan traceability-check security-scan accessibility-check review-code review-api	בודקים מאשרים את תכנית הבדיקות, משלימים בדיקות קבלה ומאמתים עקיבות	test-plan.md test-report.md traceability.md	ארכיטקט
—	שער G3 · מוכן לשחרור	כל דרישה (כולל דרישות ה-API אם היו) ממופה לבדיקה שעברה; אין ממצאי אבטחה חוסמים. מאשרים: ארכיטקט, מנטור עסקי, המשרד. הפלט: קוד בדוק בסביבת המשרד + מפרט מלא ומעודכן.					

הבהרות:

- **מפגש אחד לפאזות 1–2.** ארבעת השלבים הראשונים הם שלבים נפרדים מבחינת תוצרים ושערים, אבל הצוות עובר אותם ברצף במפגש אחד: הסוכנים מכינים טיוטות מראש, ובמפגש הצוות עונה, מכריע ומאשר שלב אחר שלב. יום הארכיטקטורה (G2) חותם את המפגש.
- **Intent** הוא הבסיס למיזם: הצורך העסקי והפתרון המוצע — מטרה, משתמשים, הערך, גבולות, מדדי הצלחה. הוא נגזר מהחבילה העסקית (שנשארת בפורמט שלה ומקושרת) והוא המסמך שכל שאר התהליך מתייחס אליו.
- **ADR — Architecture Decision Record:** קובץ קצר לכל החלטה ארכיטקטונית — הקשר, חלופות, החלטה, השלכות, מאשר. נכתב בשלב 3, ומתווסף בכל שלב שבו מתקבלת החלטה חדשה. ה-Reviewer בודק שלכל החלטה יש ADR מתועד.
- **decisions.md** — יומן החלטות המוצר והתכולה: מה בפנים ומה בחוץ, איך הוכרעה כל שאלה, מה תועדף ולמה. יחד עם ה-ADR זו שרשרת ההחלטות המלאה של המיזם.
- **State & Audit — רוחבי, נפרד מההחלטות.** STATUS.md הוא מצב התהליך: באיזה שלב ושער המיזם נמצא, מה אושר, מה פתוח. audit/ הוא יומן הביקורת: רשומה לכל ריצת שלב ולכל החלטת שער — מי אישר, מתי, מה השתנה, אילו ממצאים היו. ה-Conductor מעדכן את שניהם אוטומטית בכל שלב. decisions.md ו-ADR הם התוכן של ההחלטות; State & Audit הם העדות שהתהליך התקיים כנדרש — זה מה שמאפשר לכל גורם מפקח לראות את המיזם בלי לשאול אף אחד.
- **סיפורי המשתמש** (user-stories.md) נכתבים בשלב 2, יחד עם הדרישות — לא אחרי הארכיטקטורה: הם מה שהארכיטקטורה נענית לו, ולכן צריכים להיות מוכנים ומאושרים לפני שלב 3. תכנון הבנייה (שלב 4) והבנייה עצמה (שלב 6) מאורגנים לפי אותם סיפורים.
- **תכנון עבודה** — קובץ אחד למיזם (plan.md): סדר בניית סיפורי המשתמש, תלויות ביניהם ובעלים. המשימות לכל סיפור נכתבות בשלב 6 במפרט הסיפור.
- **חוזה API** (שלב 5) הוא שלב מותנה: רץ רק אם הארכיטקטורה מגדירה ממשקי API, ומדלגים עליו אחרת. הפלט — מפרט OpenAPI, מוק חי עם URL אמיתי, בדיקות חוזה ו-Postman collection שנוצרים אוטומטית מהמפרט — מוודא שהממשק יציב לפני שמתחילה הבנייה, כך שצוותים שונים יכולים לבנות במקביל מולו בלי להישבר.
- **תרשים הארכיטקטורה** (architecture-diagram.md) הוא תוצר לכל דבר: הסוכן מייצר אותו כקוד (Mermaid) בתוך קובץ MD, הוא עובר סקירה ואישור יחד עם מסמך הארכיטקטורה, ומתעדכן איתו בכל שינוי — כך שהתרשים לא מתיישן.

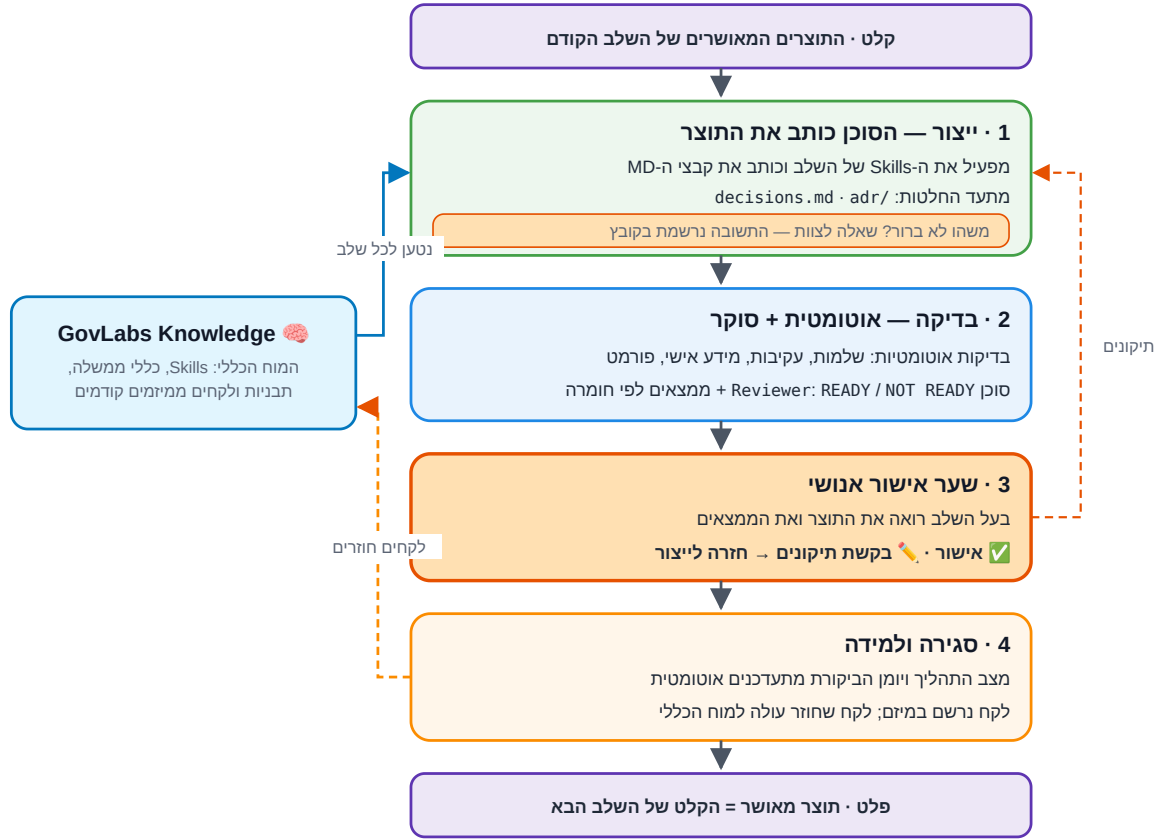
אבטחת מידע, סטנדרטים ונגישות — איפה הם נכנסים

לא שלב נפרד ולא בדיקה בסוף: הכללים נטענים מהמוח הכללי לכל שלב, והופכים לדרישה בשלב 2, לתכן בשלב 3, לקוד בשלב 6 ולראיה בדוקה בשלב 7.

היכן	מה קורה שם
רוחבי — המוח הכללי	כללי הממשלה (אבטחת מידע, פרטיות, נגישות, ענן) וסטנדרטי הפיתוח יושבים ב-GovLabs Knowledge ונטענים לכל שלב בכל מיזם — ומתורגמים לבדיקות אוטומטיות בתיקיית <code>checks/</code> שרצות לפני כל שער.
שלב 2 · דרישות	סיווג המידע נקבע כאן, ודרישות האבטחה, הפרטיות והנגישות נכתבות כדרישות עם קריטריוני קבלה — לא כהערה כללית בסוף מסמך.
שלב 3 · ארכיטקטורה	ה-Skill <code>security-privacy-requirements</code> גוזר מסיווג המידע ומכללי הממשלה את פרק האבטחה של הארכיטקטורה: הצפנה, הרשאות וזהויות, טיפול במידע אישי, רישום ובקרה. כל בחירה עם השלכת אבטחה מקבלת ADR, וה-IT המשרדי מאשר.
שלב 6 · בנייה	ה-Skill <code>coding-standards-gov</code> מלווה כל סיפור משתמש: שפות ומסגרות מאושרות, טיפול במידע אישי, רישום, טיפול בשגיאות ובדיקות יחידה. <code>review-code</code> בודק את הקוד מול אותם סטנדרטים לפני האישור.
שלב 7 · אימות	כאן נוצרת הראיה: <code>security-scan</code> (תלויות, סודות, פגיעויות ידועות), <code>accessibility-check</code> (תקן הנגישות הישראלי) והבדיקות שנגזרו מדרישות האבטחה — כולל בדיקות חוזה ה-API אם שלב 5 רץ. ממצא חוסם לא עובר את G3.

6. מה קורה בתוך כל שלב

אותו פלואו חוזר בכל שבעת השלבים: הסוכן מייצר, המערכת בודקת, אדם מאשר, והתוצר נסגר ונשמר. הצוות נכנס בשתי נקודות בלבד — שאלות הבהרה ושער האישור.



המוח הכללי נטען לכל שלב, ומתעדכן כשלקח חוזר על עצמו — הופך לכלל, ל-Skill מעודכן או לבדיקה אוטומטית חדשה. GovLabs מנהלת אותו; מנטורים מאשרים קידום לקחים.

זיכרון המיזם נשמר ברפו: לקחים ושאלות פתוחות לכל שלב (memory.md), ההחלטות (decisions.md, adr/), מצב התהליך (STATUS.md), יומן הביקורת (audit/) וההקשר הקבוע של המיזם. זה מה שמאפשר לסוכן להמשיך מחר בדיוק מאיפה שעצרנו היום.

Agent	תפקיד	מוביל	Skills
Conductor	המנצח: יודע באיזה שלב המיזם, פותח שלב, מזמן את הסוכן המתאים (כולל API Agent בשלב 5 כשנדרש), מריץ בדיקות ו-Reviewer, מציג לשער, מעדכן STATUS.md ו-audit/ , מכין את השלב הבא. מופעל בפקודה govlabs/ .	כל התהליך	run-stage gate status audit
Product	מתרגם כוונה עסקית לדרישות ברורות; שואל שאלות שאיש עסקים מבין.	2,1	business-package-to-intent write-requirements figma-to-ux
Architect	חלופות, החלטות (ADR), תרשים ארכיטקטורה, דרישות אבטחה וזיהוי ממשקי API; מכיר את אילוצי הענן הממשלתי; מתכנן את סדר הבנייה לפי סיפורי המשתמש.	4,3	architecture-doc architecture-diagram security-privacy-requirements adr gov-cloud-patterns story-plan
API	ממיר ממשק API שהוגדר בארכיטקטורה למפרט OpenAPI, מוק רץ עם URL אמיתי, בדיקות חוזה אוטומטיות ו-Postman collection.	5 (מותנה)	api-contract-spec api-mock api-tests postman-collection
Builder	מסיפור משתמש לקוד עובד עם בדיקות, לפי הסטנדרטים.	6	story-spec coding-standards-gov
Tester	תכנית בדיקות מהדרישות, בדיקות אוטומטיות, עקיבות, אבטחה, נגישות.	7	test-plan traceability-check security-scan accessibility-check

review-spec review-architecture review-code review-api	לפני כל שער	סוקר עצמאי: בודק כל תוצר מול הדרישות והכללים לפני השער ומחזיר READY / NOT READY עם ממצאים.	Reviewer
--	----------------	--	----------

Skill	היכולת שהוא מקנה
תהליך	
run-stage	מריץ שלב לפי הפלואו הקבוע: קלט → ייצור → אימות → סקירה → שער → למידה, ומעדכן את מצב המיזם.
gate	מציג לבעל השלב את התוצר, הממצאים וחוות דעת ה-Reviewer, ומתעד את ההחלטה (אישור / תיקונים) עם שם ותאריך.
status	כותב ומעדכן את STATUS.md : באיזה שלב ושער המיזם, מה אושר, מה חסר, אילו שאלות פתוחות.
audit	רושם ב-audit/ כל ריצת שלב וכל החלטת שער: מי, מתי, מה השתנה, אילו ממצאים — יומן ביקורת שאי אפשר לערוך ידנית.
מפרט	
business-package-to-intent	קורא את החבילה העסקית (Lean Canvas, בריף, Intent עסקי) וכותב ממנה את הבסיס למיזם: הצורך העסקי, הפתרון, המשתמשים, הערך, הגבולות ומדדי ההצלחה — ומסמן פערים לשאלה.
write-requirements	כותב את הדרישות הפונקציונליות והלא-פונקציונליות מתוך הכוונה העסקית, ומתעדף אותן מול צוות המיזם.
write-user-stories	כותב סיפורי משתמש עם קריטריוני קבלה בפורמט "כאשר... המערכת..." — משפטים בודקים שכל אחד מהם הופך לבדיקה, ומהם נגזר גם סדר הבנייה.
figma-to-ux	מתאר את המסכים והזרימות מעיצוב ה-Figma כטקסט, ומקשר כל מסך לסיפור המשתמש שהוא משרת.
ארכיטקטורה	
architecture-doc	כותב מסמך ארכיטקטורה: רכיבים, נתונים, אינטגרציות, אבטחה, פריסה — ומציג 2–3 חלופות עם יתרונות וחסרונות.
architecture-diagram	מייצר את תרשים הארכיטקטורה כקוד (Mermaid) בתוך architecture-diagram.md : רכיבים, זרימות מידע, אינטגרציות וגבולות אבטחה. התרשים עובר סקירה ואישור יחד עם המסמך ומתעדכן איתו.
security-privacy-requirements	גוזר מסיווג המידע ומכללי הממשלה את דרישות האבטחה והפרטיות — הצפנה, הרשאות וזהויות, טיפול במידע אישי, רישום ובקרה — וכותב אותן כסעיפים נבדקים בארכיטקטורה ובתכנית הבדיקות.
adr	מתעד החלטה ארכיטקטונית אחת בפורמט קבוע: הקשר, חלופות, החלטה, השלכות, מאשר.
gov-cloud-patterns	מכיר את תבניות הענן הממשלתי (AWS / GCP / Nimbus): אילו שירותים מאושרים, איך נראית סביבת משרד, אילו אילוצי רשת ואבטחה קיימים.
story-plan	קובע את סדר בניית סיפורי המשתמש לפי תלויות ביניהם, ומסמן את השלד ההולך הראשון (הגרסה הדקה שעוברת מקצה לקצה).
חוזי API	
api-contract-spec	הופך כל ממשק API שזוהה בארכיטקטורה למפרט OpenAPI פורמלי: נתיבים, סכמות, קודי שגיאה.

api-mock	מייצר מהמפרט שרת מוק רץ עם URL אמיתי, כך שצרכני הממשק יכולים לפתח מולו לפני שהמימוש האמיתי קיים.
api-tests	מייצר בדיקות חוזה אוטומטיות שמוודאות שהמימוש עומד במפרט — נתיבים, סכמות, קודי שגיאה.
postman-collection	מייצר מהמפרט אוסף Postman מוכן לשימוש, עם דוגמאות בקשה לכל נתיב.
בנייה	
story-spec	מייצר לסיפור משתמש את המפרט המצומצם — דרישות, תכן, משימות — מתוך המפרט והארכיטקטורה של המיזם.
coding-standards-gov	מוודא שהקוד עומד בסטנדרטים: שפות ומסגרות מאושרות, טיפול במידע אישי, רישום, טיפול בשגיאות, בדיקות יחידה.
בדיקות, אבטחה ונגישות	
test-plan	גוזר תכנית בדיקות מקריטריוני הקבלה: לכל קריטריון — בדיקה, נתוני בדיקה, תוצאה צפויה.
traceability-check	בונה את מטריצת העקיבות: כוונה → דרישה → סיפור משתמש → קוד → בדיקה, ומסמן דרישות ללא בדיקה או בדיקות ללא דרישה.
security-scan	מריץ סריקות אבטחה (תלויות, סודות, פגיעויות ידועות) ומסכם ממצאים לפי חומרה.
accessibility-check	בודק עמידה בתקן הנגישות הישראלי למסכים ולרכיבי הממשק.
סקירה	
review-api · review-code · review-architecture · review-spec	סוקרים תוצר מסוג מסוים מול הדרישות, הכללים והשלב הקודם, ומחזירים READY / NOT READY עם רשימת ממצאים לפי חומרה. סקירת הארכיטקטורה כוללת גם התאמה בין התרשים למסמך; סקירת ה-API כוללת התאמה בין המפרט, המוק והבדיקות.

כל Skill = תיקייה עם קובץ הוראות (מתי להשתמש, שלבים, תבנית הפלט, דוגמה טובה). נכתב פעם אחת ב-GovLabs Knowledge ומשתפר מכל מיזם. מהנדס כותב Skill טכני; איש עסקים יכול לכתוב Skill של תוכן (למשל "איך נראה Intent טוב") — כי זה טקסט פשוט.

8. איך עובדים

מפגש צוות

1. **לפני:** ה-Conductor פתח את השלב; הסוכן הכין טיוטה + שאלות פתוחות מסומנות בקובץ. המנחה שולח לצוות.
2. **במפגש:** הכלי פתוח על המסך. עוברים על השאלות; התשובות נכתבות ישר לקובץ; הסוכן מעדכן את הטיוטה תוך כדי. החלטות נרשמות ב- `decisions.md` או ב-ADR בזמן אמת.
3. **לפני הסיום:** הבדיקות וה-Reviewer רצים. עוברים על הממצאים. בעל השלב מאשר או מבקש סבב נוסף.
4. **אחרי:** התוצר נשמר. `STATUS.md` ו-`memory.md` מתעדכנים. ה-Conductor מכין את השלב הבא.

בין המפגשים — עבודה רציפה עם ה-AI

המפגש הוא נקודת ההכרעה, לא המקום היחיד שבו עובדים. כל חבר צוות יכול, בכל זמן, לפתוח את המיזם ולעבוד עליו עם הסוכנים: לשאול "מה סטטוס המיזם?" ולקבל את `STATUS.md` העדכני — שלב, שער, מה אושר ומה פתוח; לעבור על טיוטה ולהעיר עליה; לשאול את הסוכן למה הוחלט כך (הוא יפנה ל- `decisions.md` או ל-ADR); לבקש להריץ שוב בדיקה; או לענות על שאלה פתוחה עוד לפני המפגש. כל אינטראקציה כזו נרשמת במיזם — כך שהמפגש הבא מתחיל מנקודה מתקדמת יותר.

9. מבנה הרפו (תבנית אחת לכל מיזם)

```

my-initiative/      ← GovLabs ב-G0 נוצר מתבנית
├─ business-package/ ← (ברף, עיצובים) החבילה העסקית כפי שהתקבלה (Lean Canvas)
├─ knowledge/        ← GovLabs Knowledge: skills · rules · templates · lessons
├─ agents/           ← conductor · product · architect · api · builder · tester · reviewer
├─ context/          ← המיזם של הקבוצה: product · tech · structure · design-system · gov-roles
├─ checks/           ← בדיקות אוטומטיות לפני שער
├─ STATUS.md         ← מצב התהליך: שלב, שער, מה אושר, מה פתוח
├─ audit/            ← יומן ביקורת: כל ריצת שלב וכל החלטת שער
├─ spec/             ← (בכל תיקייה + memory.md) תוצרי שלבים 1-4, 7
│   └─ 1-intent/      intent.md
│   └─ 2-requirements/ requirements.md user-stories.md ux.md decisions.md
│   └─ 3-architecture/ architecture.md architecture-diagram.md infra.md adr/
│   └─ 4-plan/        plan.md
│   └─ 7-verify/      test-plan.md test-report.md traceability.md
├─ api/              ← הגדרת המוק · tests/ · postman.json · spec.yaml (מותנה) 5
├─ specs/<story>/     ← requirements · design · tasks: שלב 6, לכל סיפור משתמש
└─ src/ tests/ infra/ ← קוד המערכת, בדיקות, קוד תשתית

```

מיפוי התיקיות לכלי הספציפי (למשל / .kiro או / .claudio) נעשה בתבנית — המבנה הלוגי לא משתנה.

10. תוכנית יישום

שלב	מה	מתי
א. תבנית + מוח	תבנית מיזם + GovLabs Knowledge: 7 סוכנים, Skills 25, הקשר קבוע, בדיקות, מבנה תיקיות. דמו מלא בכלי הראשון (Kiro).	עכשיו (דמו יום ג' · מצגת מנטורים 10.9)
ב. פיילוט	2–3 מיזמים עוברים את התהליך המלא. כל מפגש = לקח למוח הכללי.	מחזור ראשון
ג. הרחבה	כל המיזמים בתכנית. הכשרה מדורגת (מפרט → סוכנים → Skills) לפני יום הארכיטקטורה. ליווי צמוד ל-IT המשרדים.	מחזור שני
ד. אוטונומיה	ה-Conductor והסוכנים רצים לבד (מחוץ לסביבת הפיתוח) על אותם קבצים; הצוות נשאר בשערים בלבד.	2027